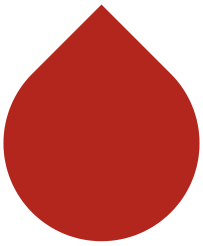
DATABASE MANAGEMENT SYSTEMS

Course Project Report



BLOOD BANK MANAGEMENT SYSTEM

A MySQL database for donors, blood stock, hospitals and blood requests

SUBMITTED BY

Roll No.	Name	GitHub
25WH1A0568	P. Sri Pregna	https://github.com/Pregna21
25WH1A0565	N. Yoshitha	https://github.com/Yoshitha2214
25WH1A05A7	S. V. Nainika	https://github.com/25wh1a05a7

B.Tech II Year · Computer Science and Engineering

November 2026

Contents

1. Introduction
- 1.1 Problem statement
- 1.2 Objectives
- 1.3 Scope

2. Software requirements

3. Entities and relationships
- 3.1 Entities
- 3.2 ER diagram
- 3.3 Relationships

4. Database schema

5. Normalization

6. Implementation
- 6.1 DDL commands
- 6.2 DML commands
- 6.3 Constraints and special queries
- 6.4 Nested and correlated subqueries
- 6.5 Aggregate functions, window functions, GROUP BY and HAVING
- 6.6 Views
- 6.7 Triggers
- 6.8 Stored procedures
- 6.9 Cursors

7. Reports
- 7.1 Blood groups with a shortage of stock
- 7.2 Eligible donors for blood groups that patients are waiting for
- 7.3 Units that expire within the next 30 days

8. Conclusion and future scope

1. Introduction

1.1 Problem statement

Hospitals depend on blood banks every day for surgeries, accident victims, mothers during childbirth and patients with conditions such as thalassemia. Blood cannot be manufactured, each component has a short shelf life, and a mistake such as issuing an expired bag or the wrong blood group can cost a life. Many blood banks still keep donor registers and stock books on paper, so it is hard to answer simple questions quickly: how many O-negative units are in stock, which bags expire this week, or which donors can be called today.

The Blood Bank Management System is a MySQL database that records donors, donations, the blood units separated from each donation, hospitals, patients and their blood requests, and the issue of units against those requests.

1.2 Objectives

- Maintain a register of donors with their blood group, contact details and donation history.
- Track every blood unit from collection to issue, with its component, expiry date and status.
- Show the live stock of each blood group in each blood bank.
- Enforce medical rules inside the database: a 90-day gap between donations, a minimum hemoglobin level of 12.5 g/dL, and no issue of expired or already issued units.
- Produce reports on shortages, eligible donors and units that are about to expire.

1.3 Scope

The system covers three blood banks, three hospitals, fifteen donors and six patients as sample data. It is designed for blood bank staff and is implemented entirely in SQL (tables, constraints, views, triggers, stored procedures and a cursor). A front-end application is outside the scope of this project.

2. Software requirements

Database server	MySQL Community Server 8.0.31 or later (needed for INTERSECT)
Client tool	MySQL Workbench 8.0 or the mysql command-line client
Diagram tool	draw.io / Graphviz for the ER diagram
Version control	Git and GitHub

3. Entities and relationships

3.1 Entities

Entity	What it stores
Donor	A person registered to donate blood
Blood Bank	A centre that collects and stores blood

Donation	One visit in which a donor gives blood at a bank
Blood Unit	A bag of one component (Whole Blood, RBC, Plasma or Platelets) separated from a donation
Hospital	A hospital that requests blood
Patient	A person admitted to a hospital who needs blood
Blood Request	A request raised for a patient for a component and number of units
Blood Issue	The handing over of one unit against a request

3.2 ER diagram

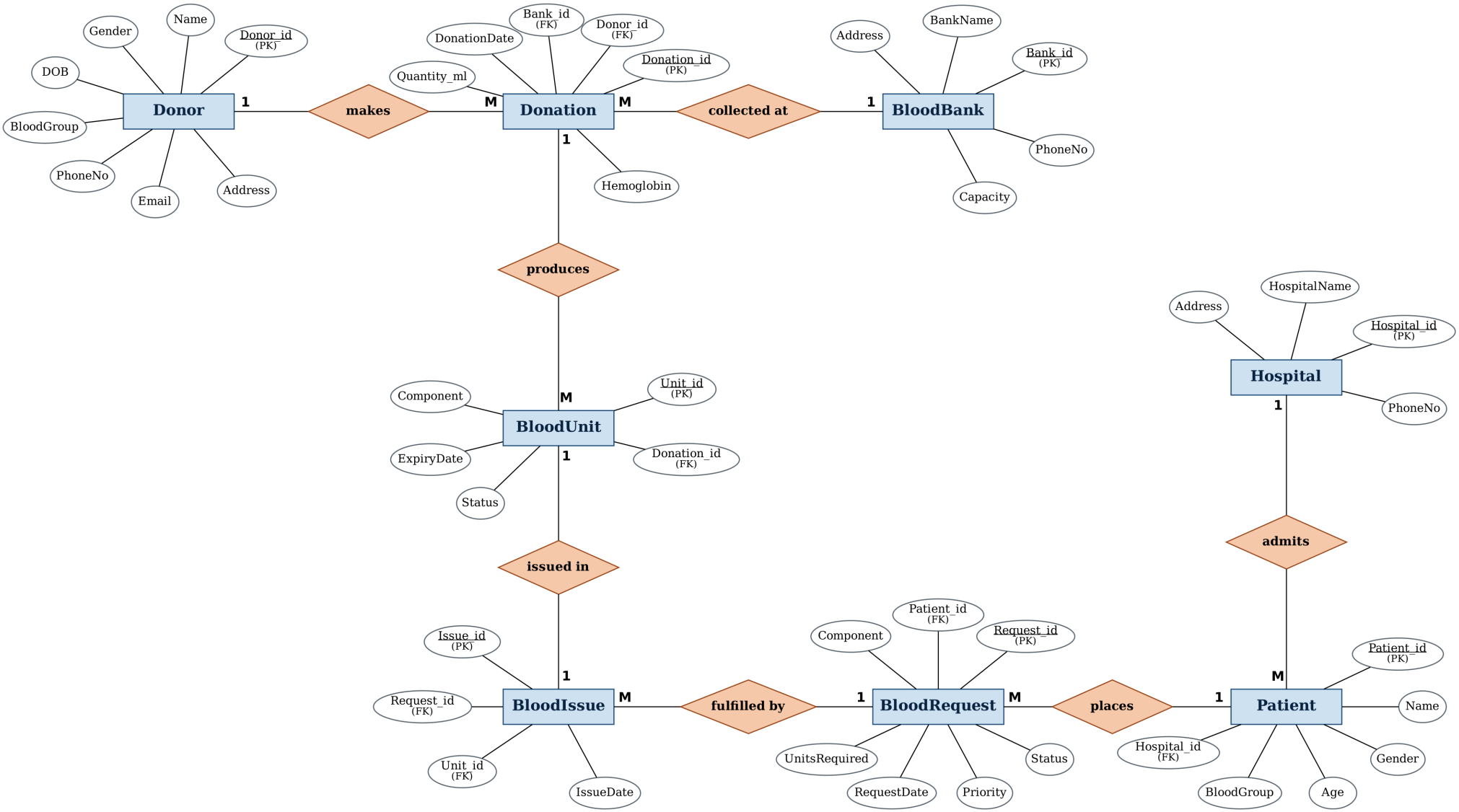


Figure 1 – Entity-Relationship diagram (Chen notation). Rectangles are entities, diamonds are relationships, ovals are attributes; underlined attributes are primary keys.

3.3 Relationships

Relationship	Entities	Cardinality	Meaning
makes	Donor – Donation	1 : M	A donor can donate many times; each donation belongs to one donor
collected at	Donation – Blood Bank	M : 1	A bank collects many donations
produces	Donation – Blood Unit	1 : M	One donation can be separated into several components
issued in	Blood Unit – Blood Issue	1 : 1	A unit can be issued only once
fulfilled by	Blood Request – Blood Issue	1 : M	A request for 2 units is fulfilled by 2 issues
places	Patient – Blood Request	1 : M	A patient can raise several requests
admits	Hospital – Patient	1 : M	A hospital admits many patients

4. Database schema

Each relation is listed with its columns, MySQL data types and constraints. PK = primary key, FK = foreign key.

4.1 DONORS

DONORS (Donor_id, Name, Gender, DOB, BloodGroup, PhoneNo, Email, Address, Location)

Column	Data type	Constraints
`Donor_id`	INT	PK
`Name`	VARCHAR(150)	NOT NULL (widened from 100 with ALTER)
`Gender`	VARCHAR(10)	CHECK Male / Female / Other
`DOB`	DATE	
`BloodGroup`	VARCHAR(3)	NOT NULL, CHECK one of 8 groups
`PhoneNo`	VARCHAR(15)	UNIQUE
`Email`	VARCHAR(100)	UNIQUE
`Address`	VARCHAR(200)	
`Location`	VARCHAR(50)	added with ALTER

4.2 BLOOD_BANKS

BLOOD_BANKS (Bank_id, BankName, Address, PhoneNo, Capacity)

Column	Data type	Constraints
`Bank_id`	INT	PK
`BankName`	VARCHAR(100)	NOT NULL
`Address`	VARCHAR(200)	
`PhoneNo`	VARCHAR(15)	
`Capacity`	INT	NOT NULL, CHECK > 0

4.3 DONATIONS

DONATIONS (Donation_id, Donor_id, Bank_id, DonationDate, Quantity_ml, Hemoglobin)

Column	Data type	Constraints
Donation_id	INT	PK
Donor_id	INT	FK → DONORS
Bank_id	INT	FK → BLOOD_BANKS
DonationDate	DATE	NOT NULL
Quantity_ml	INT	NOT NULL, CHECK 300 – 500
Hemoglobin	DECIMAL(4,1)	CHECK ≥ 12.5

4.4 BLOOD_UNITS

BLOOD_UNITS (Unit_id, Donation_id, Component, ExpiryDate, Status)

Column	Data type	Constraints
Unit_id	INT	PK
Donation_id	INT	FK → DONATIONS
Component	VARCHAR(20)	NOT NULL, CHECK 4 components
ExpiryDate	DATE	NOT NULL
Status	VARCHAR(20)	DEFAULT Available, CHECK 4 statuses

4.5 HOSPITALS

HOSPITALS (Hospital_id, HospitalName, Address, PhoneNo)

Column	Data type	Constraints
Hospital_id	INT	PK
HospitalName	VARCHAR(100)	NOT NULL
Address	VARCHAR(200)	
PhoneNo	VARCHAR(15)	

4.6 PATIENTS

PATIENTS (Patient_id, Name, Gender, Age, BloodGroup, Hospital_id)

Column	Data type	Constraints
Patient_id	INT	PK
Name	VARCHAR(100)	NOT NULL
Gender	VARCHAR(10)	
Age	INT	CHECK ≥ 0
BloodGroup	VARCHAR(3)	NOT NULL
Hospital_id	INT	FK → HOSPITALS

4.7 BLOOD_REQUESTS

BLOOD_REQUESTS (Request_id, Patient_id, Component, UnitsRequired, RequestDate, Priority, Status)

Column	Data type	Constraints
Request_id	INT	PK
Patient_id	INT	FK → PATIENTS
Component	VARCHAR(20)	NOT NULL
UnitsRequired	INT	NOT NULL, CHECK > 0 (added with ALTER)
RequestDate	DATE	NOT NULL
Priority	VARCHAR(10)	DEFAULT Normal
Status	VARCHAR(20)	DEFAULT Pending

4.8 BLOOD_ISSUES

BLOOD_ISSUES (Issue_id, Request_id, Unit_id, IssueDate)

Column	Data type	Constraints
Issue_id	INT	PK
Request_id	INT	FK → BLOOD_REQUESTS
Unit_id	INT	FK → BLOOD_UNITS, UNIQUE
IssueDate	DATE	NOT NULL

5. Normalization

The design was checked against the normal forms by writing down the functional dependencies (FDs) of every relation.

Relation	Functional dependencies	Highest normal form
DONORS	Donor_id → all other columns; PhoneNo → Donor_id; Email → Donor_id	BCNF
BLOOD_BANKS	Bank_id → BankName, Address, PhoneNo, Capacity	BCNF
DONATIONS	Donation_id → Donor_id, Bank_id, DonationDate, Quantity_ml, Hemoglobin	BCNF
BLOOD_UNITS	Unit_id → Donation_id, Component, ExpiryDate, Status	BCNF
HOSPITALS	Hospital_id → HospitalName, Address, PhoneNo	BCNF
PATIENTS	Patient_id → Name, Gender, Age, BloodGroup, Hospital_id	BCNF
BLOOD_REQUESTS	Request_id → Patient_id, Component, UnitsRequired, RequestDate, Priority, Status	BCNF
BLOOD_ISSUES	Issue_id → Request_id, Unit_id, IssueDate; Unit_id → Issue_id	BCNF

- **First normal form:** all columns are atomic. A donation that is separated into plasma and red cells is not stored as a list in one column; each component becomes its own row in BLOOD_UNITS.

- **Second normal form:** every relation has a single-column key, so no non-key column can depend on only part of a key.
- **Third normal form and BCNF:** in every FD above, the left-hand side is a candidate key, so there are no transitive dependencies. For example, an early draft stored BloodGroup and Bank_id in BLOOD_UNITS. That created the transitive dependency Unit_id → Donation_id → BloodGroup, so a mistake in one copy could make a bag show the wrong group. These columns were removed; a unit's group and bank are now found by joining DONATIONS and DONORS.

6. Implementation

This chapter lists every SQL statement of the project in the order it is run, followed by the output that MySQL prints.

6.1 DDL commands

DDL (Data Definition Language) is used to create and modify database structures.

Create database

```
CREATE DATABASE BloodBank;
USE BloodBank;
```

OUTPUT

```
Query OK, 1 row affected (0.01 sec)

Database changed
```

Create tables

```
CREATE TABLE DONORS (
    Donor_id INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Gender VARCHAR(10),
    DOB DATE,
    BloodGroup VARCHAR(3) NOT NULL,
    PhoneNo VARCHAR(15) UNIQUE,
    Email VARCHAR(100) UNIQUE,
    Address VARCHAR(200),
    CHECK (Gender IN ('Male', 'Female', 'Other')),
    CHECK (BloodGroup IN ('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-'))
);

CREATE TABLE BLOOD_BANKS (
    Bank_id INT PRIMARY KEY,
    BankName VARCHAR(100) NOT NULL,
    Address VARCHAR(200),
    PhoneNo VARCHAR(15),
    Capacity INT NOT NULL,
    CHECK (Capacity > 0)
);

CREATE TABLE DONATIONS (
    Donation_id INT PRIMARY KEY,
    Donor_id INT NOT NULL,
    Bank_id INT NOT NULL,
    DonationDate DATE NOT NULL,
    Quantity_ml INT NOT NULL,
    Hemoglobin DECIMAL(4,1),
    FOREIGN KEY (Donor_id)
        REFERENCES DONORS(Donor_id),
    FOREIGN KEY (Bank_id)
        REFERENCES BLOOD_BANKS(Bank_id),
    CHECK (Quantity_ml BETWEEN 300 AND 500),
    CHECK (Hemoglobin >= 12.5)
);

CREATE TABLE BLOOD_UNITS (
    Unit_id INT PRIMARY KEY,
    Donation_id INT NOT NULL,
    Component VARCHAR(20) NOT NULL,
    ExpiryDate DATE NOT NULL,
    Status VARCHAR(20) DEFAULT 'Available',
    FOREIGN KEY (Donation_id)
        REFERENCES DONATIONS(Donation_id),
    CHECK (Component IN ('Whole Blood', 'RBC', 'Plasma', 'Platelets')),
    CHECK (Status IN ('Available', 'Reserved', 'Issued', 'Expired'))
);

CREATE TABLE HOSPITALS (
    Hospital_id INT PRIMARY KEY,
    HospitalName VARCHAR(100) NOT NULL,
    Address VARCHAR(200),
    PhoneNo VARCHAR(15)
);

CREATE TABLE PATIENTS (
    Patient_id INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Gender VARCHAR(10),
    Age INT,
    BloodGroup VARCHAR(3) NOT NULL,
    Hospital_id INT NOT NULL,
```

```
    FOREIGN KEY (Hospital_id)
        REFERENCES HOSPITALS(Hospital_id),
    CHECK (Age >= 0)
);

CREATE TABLE BLOOD_REQUESTS (
    Request_id INT PRIMARY KEY,
    Patient_id INT NOT NULL,
    Component VARCHAR(20) NOT NULL,
    UnitsRequired INT NOT NULL,
    RequestDate DATE NOT NULL,
    Priority VARCHAR(10) DEFAULT 'Normal',
    Status VARCHAR(20) DEFAULT 'Pending',
    FOREIGN KEY (Patient_id)
        REFERENCES PATIENTS(Patient_id)
);

CREATE TABLE BLOOD_ISSUES (
    Issue_id INT PRIMARY KEY,
    Request_id INT NOT NULL,
    Unit_id INT NOT NULL UNIQUE,
    IssueDate DATE NOT NULL,
    FOREIGN KEY (Request_id)
        REFERENCES BLOOD_REQUESTS(Request_id),
    FOREIGN KEY (Unit_id)
        REFERENCES BLOOD_UNITS(Unit_id)
);
```

OUTPUT

```
Query OK, 0 rows affected (0.04 sec)
Query OK, 0 rows affected (0.03 sec)
Query OK, 0 rows affected (0.05 sec)
Query OK, 0 rows affected (0.04 sec)
Query OK, 0 rows affected (0.02 sec)
Query OK, 0 rows affected (0.03 sec)
Query OK, 0 rows affected (0.03 sec)
Query OK, 0 rows affected (0.04 sec)
```

The UNIQUE constraint on BLOOD_ISSUES.Unit_id makes sure that one blood unit can be issued only once.

Verify that all eight tables were created:

```
SHOW TABLES;
```

OUTPUT

```
+---------------------+
| Tables_in_BloodBank |
+---------------------+
| BLOOD_BANKS         |
| BLOOD_ISSUES        |
| BLOOD_REQUESTS      |
| BLOOD_UNITS         |
| DONATIONS           |
| DONORS              |
| HOSPITALS           |
| PATIENTS            |
+---------------------+
8 rows in set (0.00 sec)
```

ALTER, RENAME, TRUNCATE and DROP

```
-- Add a column
ALTER TABLE DONORS
ADD City VARCHAR(50);
```

OUTPUT

```
Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

```
-- Modify a column
ALTER TABLE DONORS
MODIFY Name VARCHAR(150) NOT NULL;
```

OUTPUT

```
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

```
-- Rename a column
ALTER TABLE DONORS
RENAME COLUMN City TO Location;
```

OUTPUT

```
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

```
-- Rename a table
RENAME TABLE BLOOD_ISSUES TO ISSUE_RECORDS;
```

OUTPUT

```
Query OK, 0 rows affected (0.02 sec)
```

```
-- Rename it back for the remaining code
RENAME TABLE ISSUE_RECORDS TO BLOOD_ISSUES;
```

OUTPUT

```
Query OK, 0 rows affected (0.02 sec)
```

```
-- Remove all rows while retaining the table structure
-- Execute only when you intentionally want to clear the table.
-- TRUNCATE TABLE BLOOD_ISSUES;

-- Drop a table (destructive; execute only when needed)
-- DROP TABLE BLOOD_ISSUES;
```

The TRUNCATE and DROP statements are commented out on purpose, so they produce no output unless you run them.

Verify the altered structure of the DONORS table:

```
DESC DONORS;
```

OUTPUT

```
+------------+--------------+------+-----+---------+-------+
| Field      | Type         | Null | Key | Default | Extra |
+------------+--------------+------+-----+---------+-------+
| Donor_id   | int          | NO   | PRI | NULL    |       |
| Name       | varchar(150) | NO   |     | NULL    |       |
| Gender     | varchar(10)  | YES  |     | NULL    |       |
| DOB        | date         | YES  |     | NULL    |       |
| BloodGroup | varchar(3)   | NO   |     | NULL    |       |
| PhoneNo    | varchar(15)  | YES  | UNI | NULL    |       |
| Email      | varchar(100) | YES  | UNI | NULL    |       |
| Address    | varchar(200) | YES  |     | NULL    |       |
| Location   | varchar(50)  | YES  |     | NULL    |       |
+------------+--------------+------+-----+---------+-------+
9 rows in set (0.00 sec)
```

6.2 DML commands

Insert 15 donors

```
INSERT INTO DONORS
(Donor_id, Name, Gender, DOB, BloodGroup, PhoneNo, Email, Address, Location)
VALUES
(1, 'Arjun', 'Male', '2000-03-14', 'O+', '9848000001', 'arjun@gmail.com', 'Street 1', 'Kukatpally'),
(2, 'Meghana', 'Female', '2001-07-22', 'A+', '9848000002', 'meghana@gmail.com', 'Street 2', 'Miyapur'),
(3, 'Rahul', 'Male', '1998-11-05', 'B+', '9848000003', 'rahul@gmail.com', 'Street 3', 'Ameerpet'),
(4, 'Sneha', 'Female', '2002-01-30', 'O-', '9848000004', 'sneha@gmail.com', 'Street 4', 'Kukatpally'),
(5, 'Vikram', 'Male', '1997-09-12', 'AB+', '9848000005', 'vikram@gmail.com', 'Street 5', 'Gachibowli'),
(6, 'Ananya', 'Female', '2003-04-18', 'A-', '9848000006', 'ananya@gmail.com', 'Street 6', 'Miyapur'),
(7, 'Kiran', 'Male', '1999-12-25', 'B-', '9848000007', 'kiran@gmail.com', 'Street 7', 'Bachupally'),
(8, 'Pooja', 'Female', '2000-06-09', 'O+', '9848000008', 'pooja@gmail.com', 'Street 8', 'Ameerpet'),
(9, 'Rohit', 'Male', '1996-02-17', 'A+', '9848000009', 'rohit@gmail.com', 'Street 9', 'Kukatpally'),
(10, 'Harika', 'Female', '2001-10-03', 'B+', '9848000010', 'harika@gmail.com', 'Street 10', 'Bachupally'),
(11, 'Sai Teja', 'Male', '2002-08-28', 'AB-', '9848000011', 'saiteja@gmail.com', 'Street 11', 'Gachibowli'),
(12, 'Lavanya', 'Female', '1999-05-11', 'O+', '9848000012', 'lavanya@gmail.com', 'Street 12', 'Miyapur'),
(13, 'Varun', 'Male', '1998-01-20', 'A+', '9848000013', 'varun@gmail.com', 'Street 13', 'Ameerpet'),
(14, 'Priya', 'Female', '2000-09-07', 'B+', '9848000014', 'priya@gmail.com', 'Street 14', 'Kukatpally'),
(15, 'Nikhil', 'Male', '1997-04-26', 'O-', '9848000015', 'nikhil@gmail.com', 'Street 15', 'Bachupally');
```

OUTPUT

```
Query OK, 15 rows affected (0.01 sec)
Records: 15  Duplicates: 0  Warnings: 0
```

```
SELECT * FROM DONORS;
```

OUTPUT

```
+----------+----------+--------+------------+------------+------------+-------------------+-----------+------------+
| Donor_id | Name     | Gender | DOB        | BloodGroup | PhoneNo    | Email             | Address   | Location   |
+----------+----------+--------+------------+------------+------------+-------------------+-----------+------------+
|        1 | Arjun    | Male   | 2000-03-14 | O+         | 9848000001 | arjun@gmail.com   | Street 1  | Kukatpally |
|        2 | Meghana  | Female | 2001-07-22 | A+         | 9848000002 | meghana@gmail.com | Street 2  | Miyapur    |
|        3 | Rahul    | Male   | 1998-11-05 | B+         | 9848000003 | rahul@gmail.com   | Street 3  | Ameerpet   |
|        4 | Sneha    | Female | 2002-01-30 | O-         | 9848000004 | sneha@gmail.com   | Street 4  | Kukatpally |
|        5 | Vikram   | Male   | 1997-09-12 | AB+        | 9848000005 | vikram@gmail.com  | Street 5  | Gachibowli |
|        6 | Ananya   | Female | 2003-04-18 | A-         | 9848000006 | ananya@gmail.com  | Street 6  | Miyapur    |
|        7 | Kiran    | Male   | 1999-12-25 | B-         | 9848000007 | kiran@gmail.com   | Street 7  | Bachupally |
|        8 | Pooja    | Female | 2000-06-09 | O+         | 9848000008 | pooja@gmail.com   | Street 8  | Ameerpet   |
|        9 | Rohit    | Male   | 1996-02-17 | A+         | 9848000009 | rohit@gmail.com   | Street 9  | Kukatpally |
|       10 | Harika   | Female | 2001-10-03 | B+         | 9848000010 | harika@gmail.com  | Street 10 | Bachupally |
|       11 | Sai Teja | Male   | 2002-08-28 | AB-        | 9848000011 | saiteja@gmail.com | Street 11 | Gachibowli |
|       12 | Lavanya  | Female | 1999-05-11 | O+         | 9848000012 | lavanya@gmail.com | Street 12 | Miyapur    |
|       13 | Varun    | Male   | 1998-01-20 | A+         | 9848000013 | varun@gmail.com   | Street 13 | Ameerpet   |
|       14 | Priya    | Female | 2000-09-07 | B+         | 9848000014 | priya@gmail.com   | Street 14 | Kukatpally |
|       15 | Nikhil   | Male   | 1997-04-26 | O-         | 9848000015 | nikhil@gmail.com  | Street 15 | Bachupally |
+----------+----------+--------+------------+------------+------------+-------------------+-----------+------------+
15 rows in set (0.00 sec)
```

Insert sample data into the remaining tables

```
INSERT INTO BLOOD_BANKS VALUES
(201, 'Lifeline Blood Bank', 'Kukatpally, Hyderabad', '04023001001', 500),
(202, 'Sanjeevani Blood Bank', 'Ameerpet, Hyderabad', '04023002002', 400),
(203, 'City Blood Centre', 'Gachibowli, Hyderabad', '04023003003', 300);
```

OUTPUT

```
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
INSERT INTO DONATIONS VALUES
(301, 1, 201, '2026-07-15', 450, 14.2),
(302, 2, 201, '2026-10-20', 450, 13.1),
(303, 3, 202, '2026-10-22', 350, 13.8),
(304, 4, 201, '2026-10-25', 450, 13.5),
(305, 5, 203, '2026-08-01', 350, 14.6),
(306, 6, 202, '2026-10-28', 450, 12.9),
(307, 8, 201, '2026-11-01', 450, 13.2),
(308, 9, 203, '2026-11-02', 350, 14.0),
(309, 1, 201, '2026-10-30', 450, 14.1),
(310, 10, 202, '2026-11-03', 350, 12.8),
(311, 12, 203, '2026-06-20', 450, 13.0),
(312, 14, 202, '2026-11-05', 450, 13.4);
```

OUTPUT

```
Query OK, 12 rows affected (0.01 sec)
Records: 12  Duplicates: 0  Warnings: 0
```

```
INSERT INTO BLOOD_UNITS VALUES
(401, 301, 'Whole Blood', '2026-08-19', 'Expired'),
(402, 302, 'RBC', '2026-12-01', 'Available'),
(403, 302, 'Plasma', '2027-10-20', 'Available'),
(404, 303, 'Whole Blood', '2026-11-26', 'Issued'),
(405, 304, 'RBC', '2026-12-06', 'Available'),
(406, 304, 'Platelets', '2026-10-30', 'Expired'),
(407, 305, 'Plasma', '2027-08-01', 'Available'),
(408, 306, 'Whole Blood', '2026-12-02', 'Reserved'),
(409, 307, 'RBC', '2026-12-13', 'Issued'),
(410, 308, 'Whole Blood', '2026-12-07', 'Available'),
(411, 309, 'Whole Blood', '2026-12-04', 'Available'),
(412, 310, 'Platelets', '2026-11-08', 'Expired'),
(413, 310, 'RBC', '2026-12-15', 'Available'),
(414, 311, 'Plasma', '2027-06-20', 'Available'),
(415, 312, 'Whole Blood', '2026-12-10', 'Available'),
(416, 312, 'Platelets', '2026-11-10', 'Available');
```

OUTPUT

```
Query OK, 16 rows affected (0.01 sec)
Records: 16  Duplicates: 0  Warnings: 0
```

```
INSERT INTO HOSPITALS VALUES
(501, 'Sunrise Hospital', 'Kukatpally, Hyderabad', '04024001001'),
(502, 'CarePlus Hospital', 'Ameerpet, Hyderabad', '04024002002'),
(503, 'Lotus Children Hospital', 'Gachibowli, Hyderabad', '04024003003');
```

OUTPUT

```
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
INSERT INTO PATIENTS VALUES
(601, 'Ramesh', 'Male', 45, 'B+', 501),
(602, 'Kavya', 'Female', 28, 'A+', 502),
(603, 'Suresh', 'Male', 60, 'O+', 501),
(604, 'Divya', 'Female', 8, 'O-', 503),
(605, 'Mahesh', 'Male', 35, 'AB+', 502),
(606, 'Geetha', 'Female', 52, 'A-', 501);
```

OUTPUT

```
Query OK, 6 rows affected (0.01 sec)
Records: 6  Duplicates: 0  Warnings: 0
```

```
INSERT INTO BLOOD_REQUESTS VALUES
(701, 601, 'Whole Blood', 1, '2026-11-04', 'Urgent', 'Fulfilled'),
(702, 602, 'RBC', 1, '2026-11-06', 'Normal', 'Fulfilled'),
(703, 603, 'Whole Blood', 2, '2026-11-08', 'Urgent', 'Pending'),
(704, 604, 'RBC', 1, '2026-11-09', 'Urgent', 'Pending'),
(705, 605, 'Plasma', 1, '2026-11-09', 'Normal', 'Pending'),
(706, 606, 'Whole Blood', 1, '2026-11-10', 'Normal', 'Pending');
```

OUTPUT

```
Query OK, 6 rows affected (0.01 sec)
Records: 6  Duplicates: 0  Warnings: 0
```

```
INSERT INTO BLOOD_ISSUES VALUES
(801, 701, 404, '2026-11-05'),
(802, 702, 409, '2026-11-06');
```

OUTPUT

```
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

Display the contents of each table to confirm the data was inserted:

```
SELECT * FROM BLOOD_BANKS;
```

OUTPUT

```
+---------+----------------------+----------------------+-------------+----------+
| Bank_id | BankName             | Address              | PhoneNo     | Capacity |
+---------+----------------------+----------------------+-------------+----------+
|     201 | Lifeline Blood Bank  | Kukatpally, Hyderabad | 04023001001 |      500 |
|     202 | Sanjeevani Blood Bank | Ameerpet, Hyderabad  | 04023002002 |      400 |
|     203 | City Blood Centre    | Gachibowli, Hyderabad | 04023003003 |      300 |
+---------+----------------------+----------------------+-------------+----------+
3 rows in set (0.00 sec)
```

```
SELECT * FROM DONATIONS;
```

OUTPUT

```
+-------------+----------+---------+--------------+-------------+------------+
| Donation_id | Donor_id | Bank_id | DonationDate | Quantity_ml | Hemoglobin |
+-------------+----------+---------+--------------+-------------+------------+
|         301 |        1 |     201 | 2026-07-15   |         450 |       14.2 |
|         302 |        2 |     201 | 2026-10-20   |         450 |       13.1 |
|         303 |        3 |     202 | 2026-10-22   |         350 |       13.8 |
|         304 |        4 |     201 | 2026-10-25   |         450 |       13.5 |
|         305 |        5 |     203 | 2026-08-01   |         350 |       14.6 |
|         306 |        6 |     202 | 2026-10-28   |         450 |       12.9 |
|         307 |        8 |     201 | 2026-11-01   |         450 |       13.2 |
|         308 |        9 |     203 | 2026-11-02   |         350 |       14.0 |
|         309 |        1 |     201 | 2026-10-30   |         450 |       14.1 |
|         310 |       10 |     202 | 2026-11-03   |         350 |       12.8 |
|         311 |       12 |     203 | 2026-06-20   |         450 |       13.0 |
|         312 |       14 |     202 | 2026-11-05   |         450 |       13.4 |
+-------------+----------+---------+--------------+-------------+------------+
12 rows in set (0.00 sec)
```

```
SELECT * FROM BLOOD_UNITS;
```

OUTPUT

```
+---------+-------------+-------------+------------+-----------+
| Unit_id | Donation_id | Component   | ExpiryDate | Status    |
+---------+-------------+-------------+------------+-----------+
|     401 |         301 | Whole Blood | 2026-08-19 | Expired   |
|     402 |         302 | RBC         | 2026-12-01 | Available |
|     403 |         302 | Plasma      | 2027-10-20 | Available |
|     404 |         303 | Whole Blood | 2026-11-26 | Issued    |
|     405 |         304 | RBC         | 2026-12-06 | Available |
|     406 |         304 | Platelets   | 2026-10-30 | Expired   |
|     407 |         305 | Plasma      | 2027-08-01 | Available |
|     408 |         306 | Whole Blood | 2026-12-02 | Reserved  |
|     409 |         307 | RBC         | 2026-12-13 | Issued    |
|     410 |         308 | Whole Blood | 2026-12-07 | Available |
|     411 |         309 | Whole Blood | 2026-12-04 | Available |
|     412 |         310 | Platelets   | 2026-11-08 | Expired   |
|     413 |         310 | RBC         | 2026-12-15 | Available |
|     414 |         311 | Plasma      | 2027-06-20 | Available |
|     415 |         312 | Whole Blood | 2026-12-10 | Available |
|     416 |         312 | Platelets   | 2026-11-10 | Available |
+---------+-------------+-------------+------------+-----------+
16 rows in set (0.00 sec)
```

```
SELECT * FROM HOSPITALS;
```

OUTPUT

```
+-------------+------------------------+----------------------+-------------+
| Hospital_id | HospitalName           | Address              | PhoneNo     |
+-------------+------------------------+----------------------+-------------+
|         501 | Sunrise Hospital       | Kukatpally, Hyderabad | 04024001001 |
|         502 | CarePlus Hospital      | Ameerpet, Hyderabad  | 04024002002 |
|         503 | Lotus Children Hospital | Gachibowli, Hyderabad | 04024003003 |
+-------------+------------------------+----------------------+-------------+
3 rows in set (0.00 sec)
```

```
SELECT * FROM PATIENTS;
```

OUTPUT

```
+------------+--------+--------+-----+------------+-------------+
| Patient_id | Name   | Gender | Age | BloodGroup | Hospital_id |
+------------+--------+--------+-----+------------+-------------+
|        601 | Ramesh | Male   |  45 | B+         |         501 |
|        602 | Kavya  | Female |  28 | A+         |         502 |
|        603 | Suresh | Male   |  60 | O+         |         501 |
|        604 | Divya  | Female |   8 | O-         |         503 |
|        605 | Mahesh | Male   |  35 | AB+        |         502 |
|        606 | Geetha | Female |  52 | A-         |         501 |
+------------+--------+--------+-----+------------+-------------+
6 rows in set (0.00 sec)
```

```
SELECT * FROM BLOOD_REQUESTS;
```

OUTPUT

```
+------------+------------+-------------+---------------+-------------+----------+-----------+
| Request_id | Patient_id | Component   | UnitsRequired | RequestDate | Priority | Status    |
+------------+------------+-------------+---------------+-------------+----------+-----------+
|        701 |        601 | Whole Blood |             1 | 2026-11-04  | Urgent   | Fulfilled |
|        702 |        602 | RBC         |             1 | 2026-11-06  | Normal   | Fulfilled |
|        703 |        603 | Whole Blood |             2 | 2026-11-08  | Urgent   | Pending   |
|        704 |        604 | RBC         |             1 | 2026-11-09  | Urgent   | Pending   |
|        705 |        605 | Plasma      |             1 | 2026-11-09  | Normal   | Pending   |
|        706 |        606 | Whole Blood |             1 | 2026-11-10  | Normal   | Pending   |
+------------+------------+-------------+---------------+-------------+----------+-----------+
6 rows in set (0.00 sec)
```

```
SELECT * FROM BLOOD_ISSUES;
```

OUTPUT

```
+----------+------------+---------+------------+
| Issue_id | Request_id | Unit_id | IssueDate  |
+----------+------------+---------+------------+
|      801 |        701 |     404 | 2026-11-05 |
|      802 |        702 |     409 | 2026-11-06 |
+----------+------------+---------+------------+
2 rows in set (0.00 sec)
```

SELECT, UPDATE and DELETE

```
-- Display all donors
SELECT * FROM DONORS;
```

OUTPUT

```
+----------+----------+--------+------------+------------+------------+-------------------+-----------+------------+
| Donor_id | Name     | Gender | DOB        | BloodGroup | PhoneNo    | Email             | Address   | Location   |
+----------+----------+--------+------------+------------+------------+-------------------+-----------+------------+
|        1 | Arjun    | Male   | 2000-03-14 | O+         | 9848000001 | arjun@gmail.com   | Street 1  | Kukatpally |
|        2 | Meghana  | Female | 2001-07-22 | A+         | 9848000002 | meghana@gmail.com | Street 2  | Miyapur    |
|        3 | Rahul    | Male   | 1998-11-05 | B+         | 9848000003 | rahul@gmail.com   | Street 3  | Ameerpet   |
|        4 | Sneha    | Female | 2002-01-30 | O-         | 9848000004 | sneha@gmail.com   | Street 4  | Kukatpally |
|        5 | Vikram   | Male   | 1997-09-12 | AB+        | 9848000005 | vikram@gmail.com  | Street 5  | Gachibowli |
|        6 | Ananya   | Female | 2003-04-18 | A-         | 9848000006 | ananya@gmail.com  | Street 6  | Miyapur    |
|        7 | Kiran    | Male   | 1999-12-25 | B-         | 9848000007 | kiran@gmail.com   | Street 7  | Bachupally |
|        8 | Pooja    | Female | 2000-06-09 | O+         | 9848000008 | pooja@gmail.com   | Street 8  | Ameerpet   |
|        9 | Rohit    | Male   | 1996-02-17 | A+         | 9848000009 | rohit@gmail.com   | Street 9  | Kukatpally |
|       10 | Harika   | Female | 2001-10-03 | B+         | 9848000010 | harika@gmail.com  | Street 10 | Bachupally |
|       11 | Sai Teja | Male   | 2002-08-28 | AB-        | 9848000011 | saiteja@gmail.com | Street 11 | Gachibowli |
|       12 | Lavanya  | Female | 1999-05-11 | O+         | 9848000012 | lavanya@gmail.com | Street 12 | Miyapur    |
|       13 | Varun    | Male   | 1998-01-20 | A+         | 9848000013 | varun@gmail.com   | Street 13 | Ameerpet   |
|       14 | Priya    | Female | 2000-09-07 | B+         | 9848000014 | priya@gmail.com   | Street 14 | Kukatpally |
|       15 | Nikhil   | Male   | 1997-04-26 | O-         | 9848000015 | nikhil@gmail.com  | Street 15 | Bachupally |
+----------+----------+--------+------------+------------+------------+-------------------+-----------+------------+
15 rows in set (0.00 sec)
```

```
-- Display only selected columns
SELECT Donor_id, Name, BloodGroup, Location
FROM DONORS;
```

OUTPUT

```
+----------+----------+------------+------------+
| Donor_id | Name     | BloodGroup | Location   |
+----------+----------+------------+------------+
|        1 | Arjun    | O+         | Kukatpally |
|        2 | Meghana  | A+         | Miyapur    |
|        3 | Rahul    | B+         | Ameerpet   |
|        4 | Sneha    | O-         | Kukatpally |
|        5 | Vikram   | AB+        | Gachibowli |
|        6 | Ananya   | A-         | Miyapur    |
|        7 | Kiran    | B-         | Bachupally |
|        8 | Pooja    | O+         | Ameerpet   |
|        9 | Rohit    | A+         | Kukatpally |
|       10 | Harika   | B+         | Bachupally |
|       11 | Sai Teja | AB-        | Gachibowli |
|       12 | Lavanya  | O+         | Miyapur    |
|       13 | Varun    | A+         | Ameerpet   |
|       14 | Priya    | B+         | Kukatpally |
|       15 | Nikhil   | O-         | Bachupally |
+----------+----------+------------+------------+
15 rows in set (0.00 sec)
```

```
-- Update a donor's location
UPDATE DONORS
SET Location = 'Madhapur'
WHERE Donor_id = 1;
```

OUTPUT

```
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Verify the update:

```
SELECT Donor_id, Name, Location
FROM DONORS
WHERE Donor_id = 1;
```

OUTPUT

```
+----------+-------+----------+
| Donor_id | Name  | Location |
+----------+-------+----------+
|        1 | Arjun | Madhapur |
+----------+-------+----------+
1 row in set (0.00 sec)
```

```
-- Delete an expired blood unit that has been discarded
DELETE FROM BLOOD_UNITS
WHERE Unit_id = 406;
```

OUTPUT

```
Query OK, 1 row affected (0.01 sec)
```

Verify the deletion:

```
SELECT * FROM BLOOD_UNITS
WHERE Status = 'Expired';
```

OUTPUT

```
+---------+-------------+-------------+------------+---------+
| Unit_id | Donation_id | Component   | ExpiryDate | Status  |
+---------+-------------+-------------+------------+---------+
|     401 |         301 | Whole Blood | 2026-08-19 | Expired |
|     412 |         310 | Platelets   | 2026-11-08 | Expired |
+---------+-------------+-------------+------------+---------+
2 rows in set (0.00 sec)
```

The DELETE statement removes unit 406, so run it only if you want that record deleted. To restore the full set of units afterwards, re-insert only the deleted row (re-running the whole INSERT INTO BLOOD_UNITS statement would fail with a duplicate-key error for the other units):

```
INSERT INTO BLOOD_UNITS VALUES
(406, 304, 'Platelets', '2026-10-30', 'Expired');
```

OUTPUT

```
Query OK, 1 row affected (0.01 sec)
Records: 1  Duplicates: 0  Warnings: 0
```

6.3 Constraints and special queries

Constraints

The tables already demonstrate primary keys, foreign keys, NOT NULL, UNIQUE, DEFAULT, and CHECK constraints.

```
-- Add a constraint
ALTER TABLE BLOOD_REQUESTS
ADD CONSTRAINT chk_units_required
CHECK (UnitsRequired > 0);
```

OUTPUT

```
Query OK, 6 rows affected (0.05 sec)
Records: 6  Duplicates: 0  Warnings: 0
```

```
-- Inspect table constraints and column definitions
SHOW CREATE TABLE DONATIONS\G
```

OUTPUT

```
*************************** 1. row ***************************
       Table: DONATIONS
Create Table: CREATE TABLE `DONATIONS` (
  `Donation_id` int NOT NULL,
  `Donor_id` int NOT NULL,
  `Bank_id` int NOT NULL,
  `DonationDate` date NOT NULL,
  `Quantity_ml` int NOT NULL,
  `Hemoglobin` decimal(4,1) DEFAULT NULL,
  PRIMARY KEY (`Donation_id`),
  KEY `Donor_id` (`Donor_id`),
  KEY `Bank_id` (`Bank_id`),
  CONSTRAINT `DONATIONS_ibfk_1` FOREIGN KEY (`Donor_id`) REFERENCES `DONORS` (`Donor_id`),
  CONSTRAINT `DONATIONS_ibfk_2` FOREIGN KEY (`Bank_id`) REFERENCES `BLOOD_BANKS` (`Bank_id`),
  CONSTRAINT `DONATIONS_chk_1` CHECK ((`Quantity_ml` between 300 and 500)),
  CONSTRAINT `DONATIONS_chk_2` CHECK ((`Hemoglobin` >= 12.5))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
1 row in set (0.00 sec)
```

Constraint names such as DONATIONS_ibfk_1 may differ slightly between MySQL versions and operating systems.

ANY

Find donors who have at least one donation with a hemoglobin level higher than some donation collected at blood bank 202.

```
SELECT Donor_id, Name
FROM DONORS
WHERE Donor_id IN (
    SELECT Donor_id
    FROM DONATIONS
    WHERE Hemoglobin > ANY (
        SELECT Hemoglobin
        FROM DONATIONS
        WHERE Bank_id = 202
    )
)
ORDER BY Donor_id;
```

OUTPUT

```
+----------+---------+
| Donor_id | Name    |
+----------+---------+
|        1 | Arjun   |
|        2 | Meghana |
|        3 | Rahul   |
|        4 | Sneha   |
|        5 | Vikram  |
|        6 | Ananya  |
|        8 | Pooja   |
|        9 | Rohit   |
|       12 | Lavanya |
|       14 | Priya   |
+----------+---------+
10 rows in set (0.00 sec)
```

The hemoglobin levels recorded at bank 202 are 13.8, 12.9, 12.8 and 13.4, so any donation above 12.8 qualifies. Harika (donor 10) is not listed because her only donation has a level of exactly 12.8.

ALL

Find donors who have a donation with a hemoglobin level higher than every donation collected at blood bank 202.

```
SELECT Donor_id, Name
FROM DONORS
WHERE Donor_id IN (
    SELECT Donor_id
    FROM DONATIONS
    WHERE Hemoglobin > ALL (
        SELECT Hemoglobin
        FROM DONATIONS
        WHERE Bank_id = 202
    )
)
ORDER BY Donor_id;
```

OUTPUT

```
+----------+--------+
| Donor_id | Name   |
+----------+--------+
|        1 | Arjun  |
|        5 | Vikram |
|        9 | Rohit  |
+----------+--------+
3 rows in set (0.00 sec)
```

A donation must exceed the highest level at bank 202 (13.8), so only the donations of 14.2, 14.1, 14.6 and 14.0 qualify.

UNION

Combine O-negative donors and O-negative patients into one list, so the blood bank can see who needs O- blood and who can give it.

```
SELECT Name, BloodGroup, 'Donor' AS Role
FROM DONORS
WHERE BloodGroup = 'O-'
UNION
SELECT Name, BloodGroup, 'Patient' AS Role
FROM PATIENTS
WHERE BloodGroup = 'O-';
```

OUTPUT

```
+--------+------------+---------+
| Name   | BloodGroup | Role    |
+--------+------------+---------+
| Sneha  | O-         | Donor   |
| Nikhil | O-         | Donor   |
| Divya  | O-         | Patient |
+--------+------------+---------+
3 rows in set (0.00 sec)
```

INTERSECT

Find donors who live in Kukatpally and have also donated blood at least once. MySQL 8.0.31 and later supports INTERSECT.

```
SELECT Donor_id
FROM DONATIONS
INTERSECT
SELECT Donor_id
FROM DONORS
WHERE Location = 'Kukatpally'
ORDER BY Donor_id;
```

OUTPUT

```
+----------+
| Donor_id |
+----------+
|        4 |
|        9 |
|       14 |
+----------+
3 rows in set (0.00 sec)
```

Arjun (donor 1) does not appear because his location was updated to Madhapur earlier in the UPDATE example.

JOIN

Display every donation with the donor name, blood group and the blood bank where it was collected.

```
SELECT
    DN.Donation_id,
    D.Name,
    D.BloodGroup,
    B.BankName,
    DN.DonationDate,
    DN.Quantity_ml
FROM DONATIONS DN
JOIN DONORS D
    ON DN.Donor_id = D.Donor_id
JOIN BLOOD_BANKS B
    ON DN.Bank_id = B.Bank_id
ORDER BY DN.Donation_id;
```

OUTPUT

```
+-------------+---------+------------+----------------------+--------------+-------------+
| Donation_id | Name    | BloodGroup | BankName             | DonationDate | Quantity_ml |
+-------------+---------+------------+----------------------+--------------+-------------+
|         301 | Arjun   | O+         | Lifeline Blood Bank  | 2026-07-15   |         450 |
|         302 | Meghana | A+         | Lifeline Blood Bank  | 2026-10-20   |         450 |
|         303 | Rahul   | B+         | Sanjeevani Blood Bank | 2026-10-22   |         350 |
|         304 | Sneha   | O-         | Lifeline Blood Bank  | 2026-10-25   |         450 |
|         305 | Vikram  | AB+        | City Blood Centre    | 2026-08-01   |         350 |
|         306 | Ananya  | A-         | Sanjeevani Blood Bank | 2026-10-28   |         450 |
|         307 | Pooja   | O+         | Lifeline Blood Bank  | 2026-11-01   |         450 |
|         308 | Rohit   | A+         | City Blood Centre    | 2026-11-02   |         350 |
|         309 | Arjun   | O+         | Lifeline Blood Bank  | 2026-10-30   |         450 |
|         310 | Harika  | B+         | Sanjeevani Blood Bank | 2026-11-03   |         350 |
|         311 | Lavanya | O+         | City Blood Centre    | 2026-06-20   |         450 |
|         312 | Priya   | B+         | Sanjeevani Blood Bank | 2026-11-05   |         450 |
+-------------+---------+------------+----------------------+--------------+-------------+
12 rows in set (0.00 sec)
```

6.4 Nested and correlated subqueries

Nested subquery

Find donors who have a donation with a hemoglobin level above the average of all donations.

```
SELECT Donor_id, Name
FROM DONORS
WHERE Donor_id IN (
    SELECT Donor_id
    FROM DONATIONS
    WHERE Hemoglobin > (
        SELECT AVG(Hemoglobin)
        FROM DONATIONS
    )
)
ORDER BY Donor_id;
```

OUTPUT

```
+----------+--------+
| Donor_id | Name   |
+----------+--------+
|        1 | Arjun  |
|        3 | Rahul  |
|        5 | Vikram |
|        9 | Rohit  |
+----------+--------+
4 rows in set (0.00 sec)
```

The average hemoglobin level is 13.55, so only donations of 13.8 and above qualify (Arjun 14.2 and 14.1, Rahul 13.8, Vikram 14.6 and Rohit 14.0).

Correlated subquery

Find donations whose quantity is greater than the average quantity collected at that same blood bank.

```
SELECT
    DN.Donation_id,
    DN.Bank_id,
    DN.Quantity_ml
FROM DONATIONS DN
WHERE DN.Quantity_ml > (
    SELECT AVG(D2.Quantity_ml)
    FROM DONATIONS D2
    WHERE D2.Bank_id = DN.Bank_id
)
ORDER BY DN.Donation_id;
```

OUTPUT

```
+-------------+---------+-------------+
| Donation_id | Bank_id | Quantity_ml |
+-------------+---------+-------------+
|         306 |     202 |         450 |
|         311 |     203 |         450 |
|         312 |     202 |         450 |
+-------------+---------+-------------+
3 rows in set (0.00 sec)
```

A correlated subquery refers to a column from the outer query. Here, D2.Bank_id = DN.Bank_id compares each donation with the average of its own bank (bank 202 averages 400 ml and bank 203 averages 383.33 ml; every donation at bank 201 is 450 ml, so none of them is above its own average).

6.5 Aggregate functions, window functions, GROUP BY and HAVING

Aggregate functions

```
SELECT
    COUNT(*) AS TotalDonations,
    SUM(Quantity_ml) AS TotalCollected_ml,
    AVG(Quantity_ml) AS AverageQuantity,
    MAX(Quantity_ml) AS MaximumQuantity,
    MIN(Quantity_ml) AS MinimumQuantity
FROM DONATIONS;
```

OUTPUT

```
+---------------+------------------+-----------------+-----------------+-----------------+
| TotalDonations | TotalCollected_ml | AverageQuantity | MaximumQuantity | MinimumQuantity |
+---------------+------------------+-----------------+-----------------+-----------------+
|             12 |              5000 |        416.6667 |             450 |             350 |
+---------------+------------------+-----------------+-----------------+-----------------+
1 row in set (0.00 sec)
```

GROUP BY

Calculate the number of donations and the total quantity collected by each blood bank.

```
SELECT
    B.Bank_id,
    B.BankName,
    COUNT(DN.Donation_id) AS Donations,
    SUM(DN.Quantity_ml) AS TotalCollected_ml
FROM BLOOD_BANKS B
JOIN DONATIONS DN
    ON B.Bank_id = DN.Bank_id
GROUP BY B.Bank_id, B.BankName
ORDER BY B.Bank_id;
```

OUTPUT

```
+---------+----------------------+-----------+-------------------+
| Bank_id | BankName             | Donations | TotalCollected_ml |
+---------+----------------------+-----------+-------------------+
|     201 | Lifeline Blood Bank  |         5 |              2250 |
|     202 | Sanjeevani Blood Bank |         4 |              1600 |
|     203 | City Blood Centre    |         3 |              1150 |
+---------+----------------------+-----------+-------------------+
3 rows in set (0.00 sec)
```

HAVING

Display blood banks that have collected more than 1500 ml of blood.

```
SELECT
    Bank_id,
    SUM(Quantity_ml) AS TotalCollected_ml
FROM DONATIONS
GROUP BY Bank_id
HAVING SUM(Quantity_ml) > 1500
ORDER BY Bank_id;
```

OUTPUT

```
+---------+-------------------+
| Bank_id | TotalCollected_ml |
+---------+-------------------+
|     201 |              2250 |
|     202 |              1600 |
+---------+-------------------+
2 rows in set (0.00 sec)
```

Window functions

Rank donations by hemoglobin level without combining their individual rows, and show the total quantity collected by the same bank alongside every row. Equal values would receive the same rank.

```
SELECT
    Bank_id,
    Donation_id,
    Hemoglobin,
    RANK() OVER (
        ORDER BY Hemoglobin DESC
    ) AS HbRank,
    SUM(Quantity_ml) OVER (
        PARTITION BY Bank_id
    ) AS BankTotal_ml
FROM DONATIONS
ORDER BY HbRank, Donation_id;
```

OUTPUT

```
+---------+-------------+------------+--------+--------------+
| Bank_id | Donation_id | Hemoglobin | HbRank | BankTotal_ml |
+---------+-------------+------------+--------+--------------+
|     203 |         305 |       14.6 |      1 |         1150 |
|     201 |         301 |       14.2 |      2 |         2250 |
|     201 |         309 |       14.1 |      3 |         2250 |
|     203 |         308 |       14.0 |      4 |         1150 |
|     202 |         303 |       13.8 |      5 |         1600 |
|     201 |         304 |       13.5 |      6 |         2250 |
|     202 |         312 |       13.4 |      7 |         1600 |
|     201 |         307 |       13.2 |      8 |         2250 |
|     201 |         302 |       13.1 |      9 |         2250 |
|     203 |         311 |       13.0 |     10 |         1150 |
|     202 |         306 |       12.9 |     11 |         1600 |
|     202 |         310 |       12.8 |     12 |         1600 |
+---------+-------------+------------+--------+--------------+
12 rows in set (0.00 sec)
```

6.6 Views

A view is a virtual table based on a SQL query.

Create a view

This view shows the real-time stock of available blood units of every blood group in each blood bank.

```
CREATE VIEW BLOOD_STOCK_VIEW AS
SELECT
    B.Bank_id,
    B.BankName,
    D.BloodGroup,
    COUNT(*) AS AvailableUnits
FROM BLOOD_UNITS U
JOIN DONATIONS DN
    ON U.Donation_id = DN.Donation_id
JOIN DONORS D
    ON DN.Donor_id = D.Donor_id
JOIN BLOOD_BANKS B
    ON DN.Bank_id = B.Bank_id
WHERE U.Status = 'Available'
GROUP BY B.Bank_id, B.BankName, D.BloodGroup;
```

OUTPUT

```
Query OK, 0 rows affected (0.02 sec)
```

Display the view

FIELD() lists the blood groups in the usual order (A+, A-, B+, B-, AB+, AB-, O+, O-).

```
SELECT * FROM BLOOD_STOCK_VIEW
ORDER BY Bank_id,
    FIELD(BloodGroup, 'A+', 'A-', 'B+', 'B-',
                      'AB+', 'AB-', 'O+', 'O-');
```

OUTPUT

```
+---------+----------------------+------------+----------------+
| Bank_id | BankName             | BloodGroup | AvailableUnits |
+---------+----------------------+------------+----------------+
|     201 | Lifeline Blood Bank  | A+         |              2 |
|     201 | Lifeline Blood Bank  | O+         |              1 |
|     201 | Lifeline Blood Bank  | O-         |              1 |
|     202 | Sanjeevani Blood Bank | B+         |              3 |
|     203 | City Blood Centre    | A+         |              1 |
|     203 | City Blood Centre    | AB+        |              1 |
|     203 | City Blood Centre    | O+         |              1 |
+---------+----------------------+------------+----------------+
7 rows in set (0.00 sec)
```

Create a view for issued blood

```
CREATE VIEW ISSUED_BLOOD_VIEW AS
SELECT
    I.Issue_id,
    P.Name AS Patient,
    H.HospitalName,
    D.BloodGroup,
    U.Component,
    I.IssueDate
FROM BLOOD_ISSUES I
JOIN BLOOD_REQUESTS R
    ON I.Request_id = R.Request_id
JOIN PATIENTS P
    ON R.Patient_id = P.Patient_id
JOIN HOSPITALS H
    ON P.Hospital_id = H.Hospital_id
JOIN BLOOD_UNITS U
    ON I.Unit_id = U.Unit_id
JOIN DONATIONS DN
    ON U.Donation_id = DN.Donation_id
JOIN DONORS D
    ON DN.Donor_id = D.Donor_id;
```

OUTPUT

```
Query OK, 0 rows affected (0.02 sec)
```

Display it:

```
SELECT * FROM ISSUED_BLOOD_VIEW
ORDER BY Issue_id;
```

OUTPUT

```
+----------+---------+-------------------+------------+-------------+------------+
| Issue_id | Patient | HospitalName      | BloodGroup | Component   | IssueDate  |
+----------+---------+-------------------+------------+-------------+------------+
|      801 | Ramesh  | Sunrise Hospital  | B+         | Whole Blood | 2026-11-05 |
|      802 | Kavya   | CarePlus Hospital | O+         | RBC         | 2026-11-06 |
+----------+---------+-------------------+------------+-------------+------------+
2 rows in set (0.00 sec)
```

Drop a view

```
DROP VIEW ISSUED_BLOOD_VIEW;
```

OUTPUT

```
Query OK, 0 rows affected (0.01 sec)
```

This removes the view, not the underlying issue records.

6.7 Triggers

A trigger automatically executes when an INSERT, UPDATE, or DELETE operation occurs on a table. For the Blood Bank Management System, the following triggers maintain an audit log of blood request changes, enforce the 90-day gap between donations, block expired or already issued units, and update the stock automatically when a unit is issued.

Create an audit table

```
CREATE TABLE REQUEST_AUDIT (
    Audit_id INT AUTO_INCREMENT PRIMARY KEY,
    Request_id INT,
    ActionType VARCHAR(20),
    OldStatus VARCHAR(20),
    NewStatus VARCHAR(20),
    AuditTime TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

OUTPUT

```
Query OK, 0 rows affected (0.03 sec)
```

INSERT trigger

```
DELIMITER //

CREATE TRIGGER after_request_insert
AFTER INSERT ON BLOOD_REQUESTS
FOR EACH ROW
BEGIN
    INSERT INTO REQUEST_AUDIT
        (Request_id, ActionType, OldStatus, NewStatus)
    VALUES
        (NEW.Request_id, 'INSERT', NULL, NEW.Status);
END //

DELIMITER ;
```

OUTPUT

```
Query OK, 0 rows affected (0.01 sec)
```

Execute the trigger by inserting a new blood request:

```
INSERT INTO BLOOD_REQUESTS
VALUES (707, 602, 'Plasma', 1, '2026-11-10', 'Normal', 'Pending');
```

OUTPUT

```
Query OK, 1 row affected (0.01 sec)
```

```
SELECT Audit_id, Request_id, ActionType, OldStatus, NewStatus
FROM REQUEST_AUDIT
ORDER BY Audit_id;
```

OUTPUT

```
+----------+------------+------------+-----------+-----------+
| Audit_id | Request_id | ActionType | OldStatus | NewStatus |
+----------+------------+------------+-----------+-----------+
|        1 |        707 | INSERT     | NULL      | Pending   |
+----------+------------+------------+-----------+-----------+
1 row in set (0.00 sec)
```

UPDATE trigger

```
DELIMITER //

CREATE TRIGGER after_request_update
AFTER UPDATE ON BLOOD_REQUESTS
FOR EACH ROW
BEGIN
    INSERT INTO REQUEST_AUDIT
        (Request_id, ActionType, OldStatus, NewStatus)
    VALUES
        (NEW.Request_id, 'UPDATE', OLD.Status, NEW.Status);
END //

DELIMITER ;
```

OUTPUT

```
Query OK, 0 rows affected (0.01 sec)
```

Execute the trigger:

```
UPDATE BLOOD_REQUESTS
SET Status = 'Cancelled'
WHERE Request_id = 707;
```

OUTPUT

```
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
SELECT Audit_id, Request_id, ActionType, OldStatus, NewStatus
FROM REQUEST_AUDIT
ORDER BY Audit_id;
```

OUTPUT

```
+----------+------------+------------+-----------+-----------+
| Audit_id | Request_id | ActionType | OldStatus | NewStatus |
+----------+------------+------------+-----------+-----------+
|        1 |        707 | INSERT     | NULL      | Pending   |
|        2 |        707 | UPDATE     | Pending   | Cancelled |
+----------+------------+------------+-----------+-----------+
2 rows in set (0.00 sec)
```

DELETE trigger

```
DELIMITER //

CREATE TRIGGER after_request_delete
AFTER DELETE ON BLOOD_REQUESTS
FOR EACH ROW
BEGIN
    INSERT INTO REQUEST_AUDIT
        (Request_id, ActionType, OldStatus, NewStatus)
    VALUES
        (OLD.Request_id, 'DELETE', OLD.Status, NULL);
END //

DELIMITER ;
```

OUTPUT

```
Query OK, 0 rows affected (0.01 sec)
```

Before deleting a request, remove any dependent records because BLOOD_ISSUES references BLOOD_REQUESTS.

Execute the trigger:

```
DELETE FROM BLOOD_ISSUES
WHERE Request_id = 707;

DELETE FROM BLOOD_REQUESTS
WHERE Request_id = 707;
```

OUTPUT

```
Query OK, 0 rows affected (0.00 sec)

Query OK, 1 row affected (0.01 sec)
```

```
SELECT Audit_id, Request_id, ActionType, OldStatus, NewStatus
FROM REQUEST_AUDIT
ORDER BY Audit_id;
```

OUTPUT

```
+----------+------------+------------+-----------+-----------+
| Audit_id | Request_id | ActionType | OldStatus | NewStatus |
+----------+------------+------------+-----------+-----------+
|        1 |        707 | INSERT     | NULL      | Pending   |
|        2 |        707 | UPDATE     | Pending   | Cancelled |
|        3 |        707 | DELETE     | Cancelled | NULL      |
+----------+------------+------------+-----------+-----------+
3 rows in set (0.00 sec)
```

The output assumes the three trigger examples are executed in sequence on a fresh audit table. MySQL assigns Audit_id automatically.

Trigger to enforce the 90-day gap between donations

A BEFORE INSERT trigger rejects a donation if the same donor has already donated within the last 90 days.

```
DELIMITER //

CREATE TRIGGER before_donation_insert
BEFORE INSERT ON DONATIONS
FOR EACH ROW
BEGIN
    IF EXISTS (
        SELECT 1
        FROM DONATIONS
        WHERE Donor_id = NEW.Donor_id
          AND ABS(DATEDIFF(NEW.DonationDate, DonationDate)) < 90
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Donor must wait 90 days between donations';
    END IF;
END //

DELIMITER ;
```

OUTPUT

```
Query OK, 0 rows affected (0.01 sec)
```

Execute the trigger by trying to record a donation for Meghana (donor 2) on 2026-11-10, only 21 days after her donation on 2026-10-20:

```
INSERT INTO DONATIONS
VALUES (313, 2, 201, '2026-11-10', 450, 13.0);
```

OUTPUT

```
ERROR 1644 (45000): Donor must wait 90 days between donations
```

Trigger to block expired or already issued units

A BEFORE INSERT trigger on BLOOD_ISSUES rejects an issue if the unit is not Available or Reserved, or if it has passed its expiry date.

```
DELIMITER //

CREATE TRIGGER before_issue_insert
BEFORE INSERT ON BLOOD_ISSUES
FOR EACH ROW
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM BLOOD_UNITS
        WHERE Unit_id = NEW.Unit_id
          AND Status IN ('Available', 'Reserved')
          AND ExpiryDate >= NEW.IssueDate
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Blood unit is expired or already issued';
    END IF;
END //

DELIMITER ;
```

OUTPUT

```
Query OK, 0 rows affected (0.01 sec)
```

Execute the trigger by trying to issue unit 401, which expired on 2026-08-19, for request 703:

```
INSERT INTO BLOOD_ISSUES
VALUES (803, 703, 401, '2026-11-10');
```

OUTPUT

```
ERROR 1644 (45000): Blood unit is expired or already issued
```

Trigger to update stock automatically

An AFTER INSERT trigger marks the issued unit as Issued and the request as Fulfilled, so the stock view always shows the correct number of units.

```
DELIMITER //

CREATE TRIGGER after_issue_insert
AFTER INSERT ON BLOOD_ISSUES
FOR EACH ROW
BEGIN
    UPDATE BLOOD_UNITS
    SET Status = 'Issued'
    WHERE Unit_id = NEW.Unit_id;

    UPDATE BLOOD_REQUESTS
    SET Status = 'Fulfilled'
    WHERE Request_id = NEW.Request_id;
END //

DELIMITER ;
```

OUTPUT

```
Query OK, 0 rows affected (0.01 sec)
```

Execute the trigger by issuing the reserved unit 408 (A- whole blood) for Geetha's request 706:

```
INSERT INTO BLOOD_ISSUES
VALUES (803, 706, 408, '2026-11-10');
```

OUTPUT

```
Query OK, 1 row affected (0.01 sec)
```

```
SELECT Unit_id, Component, Status
FROM BLOOD_UNITS
WHERE Unit_id = 408;
```

OUTPUT

```
+---------+-------------+--------+
| Unit_id | Component   | Status |
+---------+-------------+--------+
|     408 | Whole Blood | Issued |
+---------+-------------+--------+
1 row in set (0.00 sec)
```

```
SELECT Request_id, Patient_id, Status
FROM BLOOD_REQUESTS
WHERE Request_id = 706;
```

OUTPUT

```
+------------+------------+-----------+
| Request_id | Patient_id | Status    |
+------------+------------+-----------+
|        706 |        606 | Fulfilled |
+------------+------------+-----------+
1 row in set (0.00 sec)
```

Because the request status changed, the after_request_update trigger also adds a fourth row (706, UPDATE, Pending, Fulfilled) to REQUEST_AUDIT.

6.8 Stored procedures

A stored procedure is a group of SQL statements stored in the database that can be executed whenever required.

Create a procedure to display donor details

```
DELIMITER //

CREATE PROCEDURE DisplayDonors()
BEGIN
    SELECT Donor_id, Name, BloodGroup, Location
    FROM DONORS
    ORDER BY Donor_id;
END //

DELIMITER ;
```

OUTPUT

```
Query OK, 0 rows affected (0.01 sec)
```

Execute the procedure:

```
CALL DisplayDonors();
```

OUTPUT

```
+----------+----------+------------+------------+
| Donor_id | Name     | BloodGroup | Location   |
+----------+----------+------------+------------+
|        1 | Arjun    | O+         | Madhapur   |
|        2 | Meghana  | A+         | Miyapur    |
|        3 | Rahul    | B+         | Ameerpet   |
|        4 | Sneha    | O-         | Kukatpally |
|        5 | Vikram   | AB+        | Gachibowli |
|        6 | Ananya   | A-         | Miyapur    |
|        7 | Kiran    | B-         | Bachupally |
|        8 | Pooja    | O+         | Ameerpet   |
|        9 | Rohit    | A+         | Kukatpally |
|       10 | Harika   | B+         | Bachupally |
|       11 | Sai Teja | AB-        | Gachibowli |
|       12 | Lavanya  | O+         | Miyapur    |
|       13 | Varun    | A+         | Ameerpet   |
|       14 | Priya    | B+         | Kukatpally |
|       15 | Nikhil   | O-         | Bachupally |
+----------+----------+------------+------------+
15 rows in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)
```

Create a procedure to calculate the blood donated by a donor

```
DELIMITER //

CREATE PROCEDURE GetDonorDonations(
    IN input_id INT
)
BEGIN
    SELECT
        D.Donor_id,
        D.Name,
        COUNT(DN.Donation_id) AS Donations,
        COALESCE(SUM(DN.Quantity_ml), 0) AS TotalDonated_ml
    FROM DONORS D
    LEFT JOIN DONATIONS DN
        ON D.Donor_id = DN.Donor_id
    WHERE D.Donor_id = input_id
    GROUP BY D.Donor_id, D.Name;
END //

DELIMITER ;
```

OUTPUT

```
Query OK, 0 rows affected (0.01 sec)
```

Execute the procedure:

```
CALL GetDonorDonations(1);
```

OUTPUT

```
+----------+-------+-----------+-----------------+
| Donor_id | Name  | Donations | TotalDonated_ml |
+----------+-------+-----------+-----------------+
|        1 | Arjun |         2 |             900 |
+----------+-------+-----------+-----------------+
1 row in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)
```

6.9 Cursors

A cursor processes query results one row at a time. In this example, the cursor reads each donation, keeps a running total of the blood collected, and displays a summary.

Create a cursor procedure

```
DELIMITER //

CREATE PROCEDURE DisplayCollectionSummary()
BEGIN
    DECLARE finished INT DEFAULT 0;
    DECLARE d_id INT;
    DECLARE qty INT;
    DECLARE running_total INT DEFAULT 0;

    DECLARE donation_cursor CURSOR FOR
        SELECT Donation_id, Quantity_ml
        FROM DONATIONS
        ORDER BY Donation_id;

    DECLARE CONTINUE HANDLER FOR NOT FOUND
        SET finished = 1;

    DROP TEMPORARY TABLE IF EXISTS CollectionSummary;

    CREATE TEMPORARY TABLE CollectionSummary (
        Donation_id INT,
        Quantity_ml INT,
        RunningTotal_ml INT
    );

    OPEN donation_cursor;

    read_loop: LOOP
        FETCH donation_cursor INTO d_id, qty;

        IF finished = 1 THEN
            LEAVE read_loop;
        END IF;

        SET running_total = running_total + qty;

        INSERT INTO CollectionSummary
        VALUES (d_id, qty, running_total);
    END LOOP;

    CLOSE donation_cursor;

    SELECT * FROM CollectionSummary
    ORDER BY Donation_id;
END //

DELIMITER ;
```

OUTPUT

```
Query OK, 0 rows affected (0.02 sec)
```

Execute the procedure:

```
CALL DisplayCollectionSummary();
```

OUTPUT

```
+-------------+-------------+-----------------+
| Donation_id | Quantity_ml | RunningTotal_ml |
+-------------+-------------+-----------------+
|         301 |         450 |             450 |
|         302 |         450 |             900 |
|         303 |         350 |            1250 |
|         304 |         450 |            1700 |
|         305 |         350 |            2050 |
|         306 |         450 |            2500 |
|         307 |         450 |            2950 |
|         308 |         350 |            3300 |
|         309 |         450 |            3750 |
|         310 |         350 |            4100 |
|         311 |         450 |            4550 |
|         312 |         450 |            5000 |
+-------------+-------------+-----------------+
12 rows in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)
```

The final running total of 5000 ml matches the TotalCollected_ml value returned by the aggregate query.

7. Reports

The three sample queries listed in the problem statement are implemented below. They are run on the data as it stands after the trigger examples, and 2026-11-10 is used as the current date so that the output does not change from day to day (replace it with CURDATE() to use the real current date).

7.1 Blood groups with a shortage of stock

A derived table lists all eight blood groups, so that groups with no units at all are also reported. Only units that are Available and not yet expired are counted, and groups with fewer than 2 units are shown.

```
SELECT
    G.BloodGroup,
    COUNT(S.Unit_id) AS AvailableUnits
FROM (
    SELECT 'A+' AS BloodGroup UNION ALL SELECT 'A-'
    UNION ALL SELECT 'B+' UNION ALL SELECT 'B-'
    UNION ALL SELECT 'AB+' UNION ALL SELECT 'AB-'
    UNION ALL SELECT 'O+' UNION ALL SELECT 'O-'
) G
LEFT JOIN (
    SELECT U.Unit_id, D.BloodGroup
    FROM BLOOD_UNITS U
    JOIN DONATIONS DN
        ON U.Donation_id = DN.Donation_id
    JOIN DONORS D
        ON DN.Donor_id = D.Donor_id
    WHERE U.Status = 'Available'
      AND U.ExpiryDate >= '2026-11-10'
) S
    ON G.BloodGroup = S.BloodGroup
GROUP BY G.BloodGroup
HAVING COUNT(S.Unit_id) < 2
ORDER BY FIELD(G.BloodGroup, 'A+', 'A-', 'B+', 'B-',
                             'AB+', 'AB-', 'O+', 'O-');
```

OUTPUT

```
+------------+----------------+
| BloodGroup | AvailableUnits |
+------------+----------------+
| A-         |              0 |
| B-         |              0 |
| AB+        |              1 |
| AB-        |              0 |
| O-         |              1 |
+------------+----------------+
5 rows in set (0.00 sec)
```

A-, B- and AB- have no units in stock, and AB+ and O- have only one unit each, so the blood bank should arrange donation drives for these groups.

7.2 Eligible donors for blood groups that patients are waiting for

The query finds the blood groups of patients whose requests are still Pending, and lists donors of those groups who have never donated or whose last donation was at least 90 days ago.

```
SELECT
    D.Donor_id,
    D.Name,
    D.BloodGroup,
    D.PhoneNo,
    MAX(DN.DonationDate) AS LastDonation,
    DATEDIFF('2026-11-10', MAX(DN.DonationDate)) AS DaysSinceLast
FROM DONORS D
LEFT JOIN DONATIONS DN
    ON D.Donor_id = DN.Donor_id
WHERE D.BloodGroup IN (
    SELECT P.BloodGroup
    FROM BLOOD_REQUESTS R
    JOIN PATIENTS P
        ON R.Patient_id = P.Patient_id
    WHERE R.Status = 'Pending'
)
GROUP BY D.Donor_id, D.Name, D.BloodGroup, D.PhoneNo
HAVING MAX(DN.DonationDate) IS NULL
    OR DATEDIFF('2026-11-10', MAX(DN.DonationDate)) >= 90
ORDER BY D.Donor_id;
```

OUTPUT

```
+----------+---------+------------+------------+--------------+---------------+
| Donor_id | Name    | BloodGroup | PhoneNo    | LastDonation | DaysSinceLast |
+----------+---------+------------+------------+--------------+---------------+
|        5 | Vikram  | AB+        | 9848000005 | 2026-08-01   |           101 |
|       12 | Lavanya | O+         | 9848000012 | 2026-06-20   |           143 |
|       15 | Nikhil  | O-         | 9848000015 | NULL         |          NULL |
+----------+---------+------------+------------+--------------+---------------+
3 rows in set (0.00 sec)
```

The pending requests are for O+ (Suresh), O- (Divya) and AB+ (Mahesh). Arjun and Pooja (O+) and Sneha (O-) are left out because they donated less than 90 days ago, and Nikhil has never donated, so his LastDonation is NULL.

7.3 Units that expire within the next 30 days

Units are listed nearest expiry first, so that the oldest units are issued before they are wasted.

```
SELECT
    U.Unit_id,
    D.BloodGroup,
    U.Component,
    U.ExpiryDate,
    DATEDIFF(U.ExpiryDate, '2026-11-10') AS DaysLeft
FROM BLOOD_UNITS U
JOIN DONATIONS DN
    ON U.Donation_id = DN.Donation_id
JOIN DONORS D
    ON DN.Donor_id = D.Donor_id
WHERE U.Status = 'Available'
  AND U.ExpiryDate BETWEEN '2026-11-10'
                       AND DATE_ADD('2026-11-10', INTERVAL 30 DAY)
ORDER BY U.ExpiryDate, U.Unit_id;
```

OUTPUT

```
+---------+------------+-------------+------------+----------+
| Unit_id | BloodGroup | Component   | ExpiryDate | DaysLeft |
+---------+------------+-------------+------------+----------+
|     416 | B+         | Platelets   | 2026-11-10 |        0 |
|     402 | A+         | RBC         | 2026-12-01 |       21 |
|     411 | O+         | Whole Blood | 2026-12-04 |       24 |
|     405 | O-         | RBC         | 2026-12-06 |       26 |
|     410 | A+         | Whole Blood | 2026-12-07 |       27 |
|     415 | B+         | Whole Blood | 2026-12-10 |       30 |
+---------+------------+-------------+------------+----------+
6 rows in set (0.00 sec)
```

Unit 416 (platelets) expires today, so it should be used immediately; platelets last only 5 days, compared with 35 days for whole blood, 42 days for RBC and one year for frozen plasma.

8. Conclusion and future scope

The Blood Bank Management System stores the complete journey of blood, from the donor to the patient, in eight normalized tables. CHECK, UNIQUE and FOREIGN KEY constraints stop invalid data, triggers enforce the 90-day donation gap and block expired units, views give a live picture of stock and of the blood issued to patients, and the reports point staff to shortages, donors to call and units that must be used first.

The project can be extended in the following ways:

- A web or mobile front end for donors to register and book donation slots.
- Automatic SMS or email reminders to eligible donors, using the eligible-donor report.
- A scheduled MySQL EVENT that marks expired units every night instead of running the UPDATE by hand.
- Blood-group compatibility rules, so that, for example, an O- unit can be suggested for any patient in an emergency.